SECUREVAULT

SDLC LIFECYCLE MAPPING REPORT

SUBMITTED BY:

Team Cipher Syndicate

TABLE OF CONTENTS

INTRODUCTION 2
- Target End Users: 4
- Python-Based GUI 4
- C Encryption Backend 4

TECHNOLOGIES USED 5

DEVELOPMENT PROCESS 5

TESTING AND RESILIENCE 6
- Validation & Error Handling Implemented 6

ENGINEERING OBSTACLES & SOLUTIONS 7
- Before Optimization 7
- After Optimization 8

SECURITY IMPROVEMENTS 8
- System Security 9

DEPLOYMENT & PORTABILITY 9

FUTURE SECURITY ROADMAP 10

INTRODUCTION

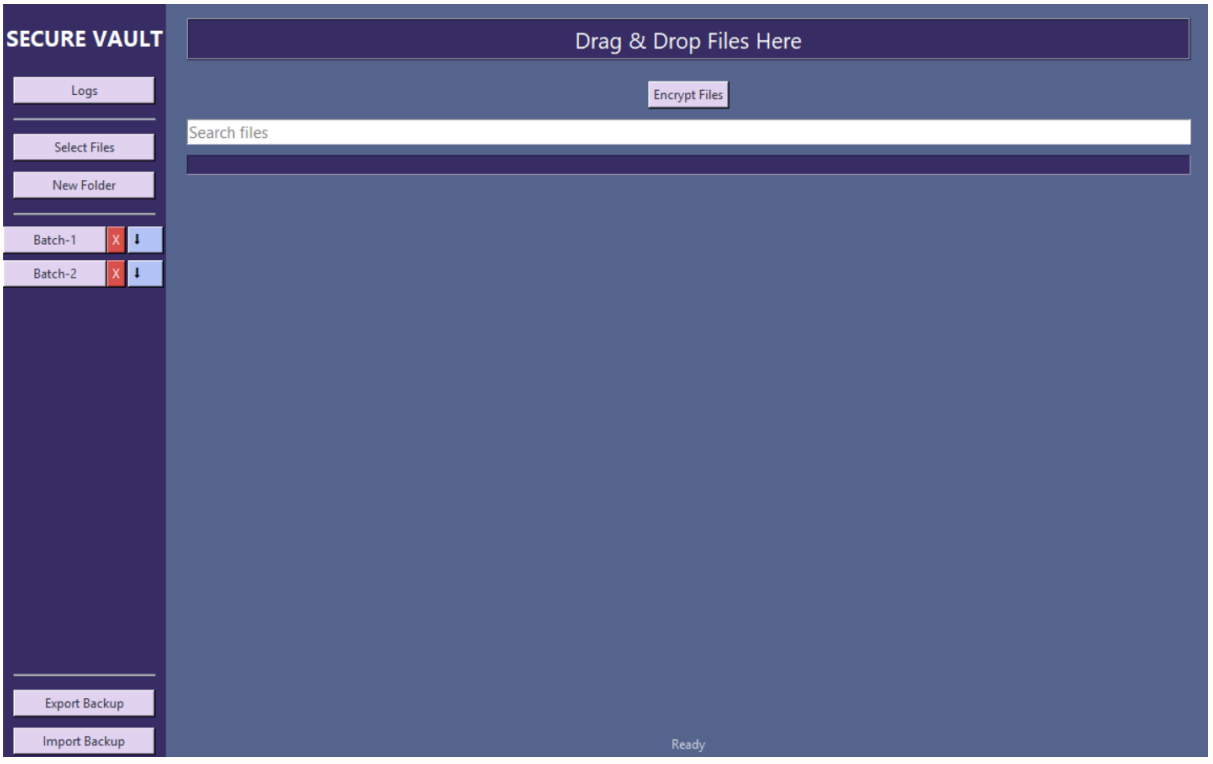
SecureVault is a desktop-based encrypted file management system developed to securely store, organize, encrypt, and protect sensitive digital files through a modern security-focused architecture.

The system combines a Python-powered graphical interface with a high-performance C/OpenSSL cryptographic backend to provide authenticated encryption, secure deletion workflows, intrusion monitoring, and encrypted backup portability.

The project was developed to address growing concerns surrounding:

- **unauthorized file access**
- **insecure local storage**
- **accidental data leaks**
- **weak password protection**
- **recoverable deleted files**

Unlike traditional file encryptors, SecureVault emphasizes both usability and layered security engineering practices.



DEVELOPMENT LIFECYCLE APPROACH

SecureVault followed a **structured iterative development** workflow focused on continuous testing, modular scalability, and security hardening.

The project lifecycle evolved through:

1. Requirement Identification
2. Secure System Design
3. Backend Cryptographic Integration
4. GUI Development & User Experience Design
5. Testing & Resilience Validation
6. Refactoring & Performance Optimization
7. Security Hardening & Deployment

This approach allowed the application to continuously evolve while **maintaining maintainability, responsiveness, and security integrity** throughout development.

REQUIREMENT ANALYSIS

Problem Statement:

Sensitive digital files are frequently exposed to risks such as **unauthorized access, accidental leaks, insecure deletion, and brute-force attacks.** Many users lack simple and secure desktop tools capable of protecting confidential files locally while maintaining usability and performance.

Target End Users:

SecureVault was designed for:

- Students
- Professionals
- Small businesses
- Privacy-conscious users
- Users storing confidential local files

The goal was to create a ***secure*** yet ***user-friendly*** desktop vault system capable of handling **encrypted storage, secure deletion, and intrusion monitoring.**

SYSTEM DESIGN

SecureVault uses a layered desktop architecture separating interface workflows from cryptographic execution.

Python-Based GUI

Responsible for:

- **User interface**
- **File management**
- **Drag-and-drop support**
- **Search filtering**
- **Logging systems**
- **Progress tracking**

C Encryption Backend

Responsible for:

- **AES-256-GCM encryption**
- **AES-256-GCM decryption**
- **PBKDF2 password key derivation**
- **File authentication and integrity verification**

The backend communicates with the Python GUI through subprocess execution. This separation improved scalability, performance, and maintainability.

TECHNOLOGIES USED

Technology	Purpose
Python	GUI and application workflows
Tkinter	Desktop graphical interface
C Programming	Cryptographic backend
OpenSSL	AES-GCM and PBKDF2 implementation
PyInstaller	Standalone executable packaging
MinGW GCC	Backend compilation

DEVELOPMENT PROCESS

The project was developed using:

- ➢ **Python**
- ➢ **Tkinter**
- ➢ **C Programming**
- ➢ **OpenSSL Cryptography Library**

The key features implemented during development included:

- ★ **Encrypted vault folders**
- ★ **Password-protected access**
- ★ **Drag-and-drop uploads**
- ★ **Live file search**
- ★ **Intrusion detection logging**
- ★ **Secure file shredding**
- ★ **Encrypted backup export and recovery**
- ★ **Real-time encryption progress tracking**

TESTING AND RESILIENCE

SecureVault was tested against **multiple edge cases** and **unexpected user interactions** to improve system stability and security. The following are some examples:

Validation & Error Handling Implemented

- ★ Empty file selection prevention
- ★ Incorrect password detection
- ★ Temporary vault lockout after repeated failed attempts
- ★ Missing backend executable handling
- ★ Secure deletion failure handling
- ★ Corrupted output cleanup after failed decryption
- ★ Tampered file detection using AES-GCM authentication tags
- ★ Thread-safe GUI updates during encryption

The application uses **message-based feedback** systems to safely notify users instead of crashing during invalid operations.

ENGINEERING OBSTACLES & SOLUTIONS

Throughout the development, we faced many challenges. Here are a few select obstacles we encountered and how we dealt with them:

Challenge	Solution
GUI freezing during encryption	Implemented threaded encryption workflows
Growing monolithic codebase	Refactored into modular reusable components
Weak password resistance	Integrated PBKDF2-HMAC-SHA256 derivation
Unauthorized vault access attempts	Added intrusion detection and temporary lockouts
Recoverable deleted files	Implemented secure file shredding

UI responsiveness during background tasks	Added thread-safe Tkinter updates
Deployment complexity	Packaged application into standalone executable

---

REFACTORING AND OPTIMIZATION

During development, the project codebase became increasingly difficult to maintain due to a **large monolithic application structure.**

Before Optimization

- ➔ Single 1000+ line app.py file
- ➔ Tightly coupled functionality
- ➔ Difficult feature scaling and debugging
- ➔ GUI freezing during large encryption operations
- ➔ Manual dependency setup required to run this application
- ➔ Slower cryptographic processing workflow

After Optimization

The application was refactored into **modular reusable components.**

```
features/
├── logs.py
├── security.py
├── downloads.py
├── preview.py
├── password_ui.py
├── search.py
└── backup.py
```

Additional optimizations included:

- ★ Background threaded encryption to prevent GUI freezing
- ★ Real-time progress bar synchronization
- ★ Dynamic live file filtering
- ★ Organized color-coded logging systems
- ★ Improved scalability and maintainability

- ★ Standalone executable packaging for improved portability across systems

These improvements significantly increased maintainability, scalability, responsiveness, and code readability.

SECURITY IMPROVEMENTS

Several security-focused improvements were implemented throughout the project lifecycle.

Cryptographic Security

- ★ AES-256-GCM authenticated encryption
- ★ PBKDF2-HMAC-SHA256 key derivation
- ★ randomized cryptographic salts
- ★ randomized initialization vectors (IVs)
- ★ authentication tag verification
- ★ tamper detection during decryption

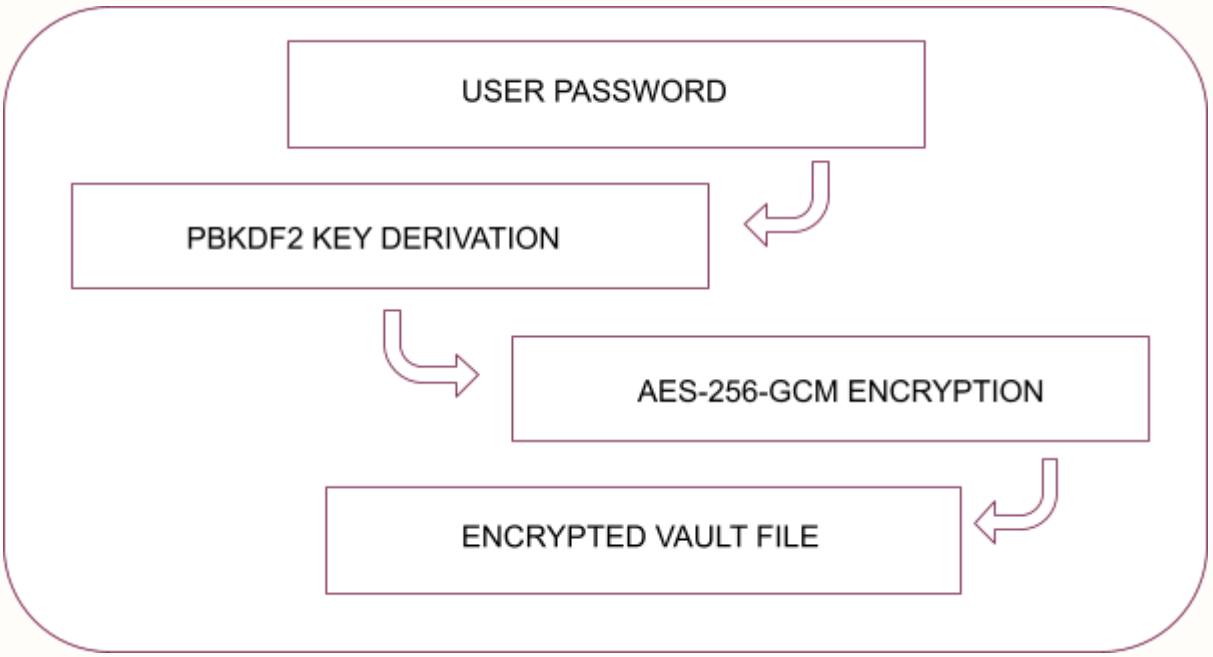
System Security

- ★ intrusion detection logging
- ★ temporary vault lockouts
- ★ secure file shredding
- ★ password strength analysis
- ★ encrypted backup portability

These additions strengthened protection against:

- ➔ brute-force attacks
- ➔ encrypted data leaks
- ➔ unauthorized vault access
- ➔ forensic file recovery attempts

SECURITY WORKFLOW



DEPLOYMENT & PORTABILITY

SecureVault was packaged into a standalone desktop executable using PyInstaller to improve deployment portability across systems.

This allowed the application to:

- **run without manual Python installation**
- **simplify deployment workflows**
- **improve accessibility for non-technical users**
- **reduce dependency configuration complexity**

The cryptographic backend was compiled separately using MinGW GCC and integrated directly into the packaged desktop application.

FUTURE SECURITY ROADMAP

Potential future improvements include:

- **biometric authentication**
- **secure cloud synchronization**
- **multi-device encrypted vault support**

- hardware-backed encryption integration
- AI-assisted intrusion analysis
- encrypted secure file sharing
- cross-platform desktop deployment

These upgrades would further improve scalability, accessibility, and advanced security capabilities.

CONCLUSION

SecureVault evolved through continuous testing, refactoring, and optimization, the project demonstrates practical implementation of secure software engineering principles while maintaining an accessible and responsive user experience.